

UVSim Software Requirement Specification – Final

Functional Requirements:

1. The system shall be able to receive and read a file supplied by the user that is a .txt file
2. The system shall be able to store user submitted instruction values from the given .txt file line by line into a location in a memory class, starting from memory location 00 and incrementing by one.
3. The system shall execute instructions sequentially within a loop in ascending memory-location order, incrementing the program counter by one to track which step is next to execute.
4. The system shall re-prompt the user if the input is invalid until the user enters a valid input or terminates the program. Valid inputs should be within the range of +999999 and -999999 and be an integer.
5. The system shall be able to handle overflows via truncation to keep the last 4 digits of numbers that are greater than 999999 or less than –999999.
6. The system shall be able to execute a SUBTRACT instruction, it shall subtract the word at the operand memory location from the accumulator (opcode 31).
7. The system shall be able to execute an ADD instruction, it shall add the word at the operand memory location to the accumulator (opcode 30).
8. The system shall be able to execute a MULTIPLY instruction, it shall multiply the accumulator by the word at the operand memory location (opcode 33).
9. The system shall be able execute a DIVIDE instruction, it shall divide the accumulator by the word at the operand memory location (opcode 32).
10. The system shall stop running the program when it hits a HALT instruction (opcode 43). Stopping execution of the execution loop.
11. The system shall set the program counter to a new address when it hits a BRANCH instruction (opcode 40).
12. The system shall be able to branch to a specific memory location if the accumulator is a negative value (opcode 41).

13. The system shall be able to branch to a specific memory location if the accumulator value is zero (opcode 42).
14. The system shall be able to execute a LOAD instruction, it shall copy the word at the operand memory location into the accumulator (opcode 20).
15. The system shall be able to execute a STORE instruction, it shall copy the accumulator value into the operand memory location (opcode 21).
16. The system shall be able to execute a READ instruction, it shall prompt the user for a word and store the entered word at the specific operand memory location given by the instruction file (opcode 10).
17. The system shall be able to execute a WRITE instruction, it shall display the word stored at the specific operand memory location given by the instruction file (opcode 11).
18. The system shall have unit tests that fail and succeed based on expected values in a spreadsheet and provide different error messages when a test fails.
19. The system shall have 250 memory locations that can be accessed in a memory class.
20. The system shall be able to allow the user to change the systems color scheme of their choosing with a primary and a secondary color.
21. The system shall be able to load and save a file from any directory on the computer.
22. The system shall be able to convert 4-digit legacy word txt files and convert them into 6-digit new file formats.
23. The system shall be able to load the file into a gui that the user can then see the file loaded and edit the file with copy, paste, and cut commands.
24. The system shall be able to have multiple different files loaded in different tabs, allowing only one to run at a time.

Non-Functional Requirements:

1. Every interactive control in the GUI shall display a text label, identifying its function.

2. The system shall run using just Python 3, without needing the user to install any extra packages.
3. The system will be able to run and execute the provided file in 2 seconds with valid inputs.